Applications using Stack

Stack :

We have discussed the **Abstract Data Type** stack. We have said that it is a data structure where all operations happen at one end – called the stack top. We can push an element to the stack, pop an element, peek the top of the stack, check for stack being empty and check for stack being full. Nothing else is supported by this type.

There are many applications which require this data structure. In formal languages, we talk about a set of languages which require such a data structure – called push down automaton. Almost all programming language features belong to this category. These languages are called context free languages. You will study sometime later in your curriculum.

Well Formed Parentheses:

Let us discuss one example in this document. How to find whether a given expression has well performed parentheses? For example,

(3 + 4)

((5 + 6) * (7 + 8))

These are well performed. For making our life easier, we can ignore all other characters other than parentheses (and).

So, the above expression can be written as

()

(()())

Clearly, these are not well formed.

(()

())

So, one of the requirements is that the number of left parentheses should be equal to the number of right parentheses. Is that sufficient? If so, we may not require a stack at all.

What if the parentheses are like this?

)(

())(())

These are not well formed. A right parenthesis occurs before the left one!

So, what is the algorithm we shall follow to check the well formedness of the parentheses?

- Initialize the stack

  ```
  stack_t mystack;
  init_stack(&mystack);
  ```

- Read the given expression. This is basically a string.

  ```
  char str[MAXSIZE + 1];
  scanf("%s", str);
  ```

- Walk through the string character by character
  - If the character is a left parenthesis, put a marker that we have seen a left parenthesis and that we can expect a corresponding right parenthesis.

    Push a place holder onto the stack
  - If the character is a right parenthesis, pop the place holder from the stack

    If the stack is empty, then we have got a right parenthesis before the left. The expression is not well formed. Exit the loop and give the message that the expression is not well formed.

```
int is_valid = 1;
char ch; char *s = str;
while( is_valid && (ch = *s++) != '\0')
{
        if(ch == '(')
        {
                push(&mystack, 'x'); // some placeholder
        }
        else if(ch == ')')
        {
                if(! is_empty(&mystack))
                        pop(&mystack);
                else
                        is_valid = 0;
        }
}
```

- At this point, the stack should be empty indicating that for every left parenthesis encountered, we have also encountered the corresponding right parenthesis in order.
- If the stack is empty, give the message that the expression is well formed else give the message that the expression is not well formed.

```
if(is_valid && is_empty(&mystack))
{
        printf("Well formed expression\n");
}
else
{
        printf("not well formed expression\n");
}
```

Please observe that we use the stack like any other type we know. We create a variable of type stack like the way we would have created an int variable. We initial the stack. We then perform the

valid operations on the stack the way we would perform addition, subtraction on int. We do not know how addition of int is carried out in the computer and we do not worry about it. Similarly we do not have to know how the operations on the stack are performed. We are not required to know them to use the stack.

To use a type, we should know the interface and we do not have know the implementation.

Well formed brackets:

In school days, we have learnt 4 types of brackets – vinculum, simple brackets or parentheses, flower brackets or curly brackets, big or square brackets. We cannot type vinculum. What if an expression contains all these types of brackets? How do we check whether such an expression is well formed?

For example,

(){}[]

([]{})

([{}])

are all well formed.

([)]

([]])

(][)

are not well formed.

We can do that by using the stack.

In that example whenever we encountered a left parenthesis, we pushed a place holder on the stack, and we removed that whenever we encountered a right parenthesis.

This is not enough when we have multiple types of brackets. If we encounter a right], we should have encountered a left [before this.

So, for every type of left bracket we encounter, we should put a different type of marker on the stack. When we get a right bracket, we should be able to pop and match whether the marker for the corresponding left bracket was on the stack.

```
char str[MAXSIZE + 1];
char *s = str;
int is_valid = 1;
char ch;
while( is_valid && (ch = *s++) != '\0')
{
        if(ch == '(')
        {
```

```
                push(&mystack, ')');
        }
        else if(ch == '[')
        {
                push(&mystack, ']');
        }
        else if(ch == '{')
        {
                push(&mystack, '}');
        }
        else if(ch == ')' || ch == ']' || ch == '}')
        {
                if(! is_empty(&mystack))
                {
                        if(ch != pop(&mystack))
                        {
                                is_valid = 0;
                        }
                }
                else
                        is_valid = 0;
        }
}
```

Let us observe the change in the code compared to the previous one. As we should distinguish the type of the bracket we require a different type of placeholder for each. We may as well choose the corresponding right bracket to push into the stack.

When we encounter a right bracket, the stack should not be empty. If it is empty, the expression is not valid. The top character should match the right bracket in the stack. Otherwise the expression is not valid.

Once the expression has been complete processed, the stack should be empty for the expression to be valid.

```
if(is_valid && is_empty(&mystack))
{
        printf("Well formed expression\n");
}
else
{
        printf("not well formed expression\n");
}
```

Observe that we have used the stack without knowing whether the stack is implemented using an array or a list. Are you not thrilled?